

AI Assignment: Handwritten Digit Recognition (Image Classification)

Level: AI Engineer-Fresher / 1–2 Years Experience)

Estimated Time: 24 hours

Tools Allowed: Python, NumPy, Pandas, scikit-learn, Matplotlib, Seaborn, flask, any python library.

Tools NOT Allowed: Any pre-trained models, LLMs, or Managed Cloud Platforms (MCPs)

Objective

Develop a machine learning pipeline to classify grayscale images of handwritten digits (0–9). Implement classical ML models from scratch, carry out image preprocessing and normalization, and evaluate the models using metrics such as accuracy and confusion matrix.

Outcome:

The model should predict the digit (0–9) for each input image. It should also provide evaluation results, including overall accuracy and a confusion matrix showing correct and incorrect predictions per digit. Optionally, pre-processed images and a brief analysis of common misclassifications can be included.

Dataset

- Dataset: MNIST subset in **CSV format** (flattened 28x28 images → 784 features per row).
- Each row contains:
 - label → digit (0–9)
 - pixel0, pixel1, ..., pixel783 → grayscale pixel values (0–255)

Example row:

5,0,0,12,...,0

You can download a sample CSV from kaggle

Assignment Tasks

Task 1: Data Loading and Exploration

1. Load the CSV file using Pandas.
 2. Explore dataset statistics:
 - Total number of samples
 - Class distribution
 - Display 5–10 sample images with their labels (reshape pixels to 28x28)
 3. Check for missing values.
-

Task 2: Data Preprocessing

1. Normalize pixel values to the range **0–1**.
 2. Split the dataset into **training** and **testing** sets (e.g., 80/20 split).
 3. Optional: Use **PCA** to reduce dimensionality (not mandatory, but recommended for improving SVM performance).
-

Task 3: Model Implementation

Train and evaluate **three classical ML models** from scikit-learn:

1. **K-Nearest Neighbors (KNN)**
 - Tune k (number of neighbors)
 - Evaluate accuracy on the test set
2. **Support Vector Machine (SVM)**
 - Use RBF or linear kernel
 - Tune C and gamma parameters
3. **Decision Tree**
 - Tune max_depth and min_samples_split

Important: Do **not** use pre-trained models or neural networks. Only classical ML algorithms are allowed.

Task 4: Model Evaluation

1. Compute **accuracy** for each model.
 2. Generate a **confusion matrix** for each model.
 3. Visualize 5–10 misclassified images and discuss why they might be misclassified.
-

Task 5: Reporting

1. Compare the performance of KNN, SVM, and Decision Tree.
 2. Write a short analysis (100–200 words) discussing:
 - Which model performed best and why
 - Observations on misclassified digits
 - Suggestions to improve performance (e.g., feature scaling, dimensionality reduction)
-

Expected Output

- Printed accuracy for each model
 - Confusion matrices visualized with heatmaps
 - Sample images of misclassified digits with predicted vs actual labels
 - Short report summarizing insights
-

Skills Tested

- Data loading and exploration with Pandas
 - Image reshaping and visualization with Matplotlib
 - Normalization and preprocessing of features
 - Implementation and hyperparameter tuning of classical ML models
 - Evaluation metrics (accuracy, confusion matrix)
 - Critical thinking and reporting
-

Bonus (Optional, for extra credit)

1. Implement a **voting ensemble** combining the three models to see if ensemble improves accuracy.
 2. Apply **PCA** and check the effect on SVM and KNN performance.
-

Submission Guidelines

- Create a flow diagram of execution of this assignment technically.
- Submit a single **Python notebook (.ipynb)** with:
 - All code cells executed
 - Implement at least one classical ML model (e.g., K-Nearest Neighbors, Decision Tree, or SVM) **from scratch** without using ready-made ML libraries for training.
 - Outputs visible(folder of Input and output images)
 - Comments explaining each step (imp)
- Include a **PDF notebook with executed code and** summarizing observations and conclusions.